



PROJECT REPORT

TITLE: Design and Implementation of a Database Management System
for Car Rental Reservation and Vehicle Return Management System



NAME: Sansita.M

ROLL NO. : 25WU0102244

COURSE NAME: Database Management Systems

FACULTY NAME: Dr Kiran Mayee Adavala

ACADEMIC YEAR: 2025-2029

B. TECH AI ML Whales

ABSTRACT

The Car Rental Management System is a web-based database application developed to manage the major activities involved in a car rental process. The system is designed to store and organize customer details, vehicle information, reservations, rental transactions, vehicle returns and payment records in a centralized MySQL database.

The main purpose of the project is to reduce the difficulties associated with maintaining car rental records manually. In a traditional system, customer information, vehicle availability, reservations, rental details and payment information may be maintained separately, which can result in duplication, inconsistency and difficulty in retrieving information. The proposed system provides a structured solution by using a relational database and a user-friendly web interface.

The application is developed using **HTML and CSS** for the frontend and **Python Flask** for the backend. MySQL is used as the database management system, while MySQL Connector/Python is used to establish communication between the Flask application and the database. The system provides CRUD operations such as inserting, viewing, updating and deleting records.

The database consists of six main tables: Customer, Vehicle, Reservation, Rental, Vehicle Return and Payment. Primary keys and foreign keys are used to maintain relationships between the tables and preserve referential integrity.

The project demonstrates how database concepts such as relational schema design, normalization, constraints, SQL commands, CRUD operations and foreign key relationships can be implemented in a practical web-based application.

INTRODUCTION

Car rental services involve managing different types of information such as customer details, vehicle information, reservations, rental transactions, vehicle returns, and payments. Managing all these records manually can become difficult as the number of customers and vehicles increases. It may also lead to errors, duplicate records, and difficulty in finding or updating information.

A Database Management System (DBMS) provides an organized solution for managing this information. In the proposed Car Rental Management System, related data is stored in separate tables and connected using relationships. Primary keys and foreign keys are used to maintain the connection between customers, vehicles, reservations, rentals, returns, and payments. This helps maintain data consistency and reduces unnecessary duplication.

The system is designed to manage the complete car rental process, starting from customer registration and vehicle selection to reservation, rental, vehicle return, and payment recording. Vehicle availability can also be maintained based on rental and return information.

A web interface is provided to make the system easier to use. Users can perform basic database operations such as adding, viewing, updating, and deleting records without directly entering SQL commands every time. This makes the system more convenient and demonstrates the practical use of DBMS concepts in a real-world car rental application.

Overall, the proposed system aims to provide a simple, organized, and efficient way to manage car rental records while reducing manual work and improving data accuracy.

PROBLEM STATEMENT

Managing car rental records manually can become difficult when the number of customers, vehicles, and rental transactions increases. Maintaining customer details, vehicle information, reservations, rental records, return details, and payment information separately can lead to confusion and errors.

One of the major challenges is tracking vehicle availability. Without a proper system, it may be difficult to know whether a vehicle is available, reserved, currently rented, or already returned. Reservation and rental details may also become disorganized, making it harder to identify the correct customer or vehicle associated with a transaction.

Another problem is maintaining consistency between related records. For example, a payment should be connected to the correct rental, and a return should be associated with the corresponding rental transaction. Manual management can also result in duplicate data, outdated information, and difficulties while searching, updating, or deleting records.

Therefore, a centralized database system is required to organize all car rental information in one place. The proposed Car Rental Management System addresses these problems by storing related information in structured tables and maintaining relationships between them. A web interface is also provided to make common database operations such as adding, viewing, updating, and deleting records easier for users.

The system aims to reduce manual work, improve data accuracy, maintain consistency between related records, and make car rental information easier to manage and access.

OBJECTIVES

The major objectives of the proposed Car Rental Management System are:

1. To manage customer information efficiently by storing and organizing customer details in a structured manner, making the information easy to access and update whenever required.
2. To maintain vehicle details and availability by storing information about vehicles and their current status, helping users identify which vehicles are available, reserved, or rented.
3. To manage customer reservations by recording reservation details and maintaining a proper connection between customers and the vehicles they wish to rent.
4. To maintain rental transaction records by storing details of each rental and linking the transaction with the corresponding customer and vehicle.
5. To record vehicle return details and late fees by maintaining information about returned vehicles and recording additional charges when a vehicle is returned late.
6. To maintain payment information by recording the payment details related to each rental transaction and keeping the financial records organized.
7. To provide CRUD operations through a web interface so that users can easily create, view, update, and delete database records without having to enter SQL commands for every operation.
8. To reduce manual work and data duplication by storing information in a centralized relational database and avoiding unnecessary repetition of records.
9. To maintain relationships between different database tables using primary keys and foreign keys so that related customer, vehicle, reservation, rental, return, and payment information remains consistent.
10. To provide centralized access to rental information by bringing all important car rental records together in one system, making the overall rental management process simple, organized, and efficient.

SCOPE OF THE PROJECT

The scope of the proposed Car Rental Management System covers the major activities involved in managing customers, vehicles, reservations, rental transactions, vehicle returns, and payments. The system provides a centralized database where rental information can be stored, maintained, and accessed through a simple web interface.

Customer Management

The system stores important customer details such as Customer ID, name, phone number, email address, and driving license number. These records can be added, viewed, updated, and deleted whenever required. Customer information can also be connected with their reservations and rental transactions.

Vehicle Management

The system maintains vehicle details including Vehicle ID, brand, model, vehicle type, rent per day, and availability status. The vehicle status helps identify whether a vehicle is available, reserved, or rented, making vehicle management easier.

Reservation and Rental Management

The system records customer reservations using Reservation ID, Customer ID, Vehicle ID, start date, end date, and reservation status. It also maintains rental transactions using Rental ID, reservation details, rent date, due date, and total amount. This helps maintain a clear connection between customers, vehicles, reservations, and rentals.

Vehicle Return Management

The system records vehicle return details such as Return ID, Rental ID, return date, vehicle condition, and late fee. This information helps maintain proper return records and supports updating the vehicle's availability after it is returned.

Payment Management

The system maintains payment information related to rental transactions. Payment ID, Rental ID, payment date, amount, and payment method are stored to provide an organized record of customer payments.

Database Operations and Web Interface

The project provides basic CRUD operations such as creating, viewing, updating, and deleting records through a web-based interface. This reduces the need to manually enter SQL commands for common database operations and makes the system easier to use.

Database Relationships

Primary keys and foreign keys are used to establish relationships between customers, vehicles, reservations, rentals, returns, and payments. This helps maintain data consistency, reduce duplication, and preserve data integrity.

Overall, the project provides a centralized and organized solution for managing the main activities of a car rental service. It demonstrates the practical application of DBMS concepts such as relational tables, keys, relationships, CRUD operations, and data management in a real-world system.

SOFTWARE AND HARDWARE REQUIREMENTS

The Car Rental Management System requires a basic computer system with suitable software tools for developing the web application, managing the relational database, and executing the system. The project can be developed and executed using the following software and hardware configuration.

Software Requirements

- Operating System: Windows 10/11 or equivalent operating system
- Visual Studio Code: Used for writing, editing, and managing the source code of the web application.
- Python: Used for developing the backend of the application and implementing the server-side functionality.
- Flask: A Python web framework used for developing the backend server and connecting the web interface with the database.
- MySQL: A relational database management system used for storing and managing customer, vehicle, reservation, rental, return, and payment information.
- HTML5: Used for creating the basic structure and content of the web-based user interface.
- CSS3: Used for designing and formatting the web interface and improving its overall appearance and usability.
- JavaScript: Used for implementing interactive features and handling client-side functionality in the web application.
- MySQL Connector/Python: Used to establish a connection between the Python Flask application and the MySQL database.
- Web Browser: Google Chrome, Microsoft Edge, or any other modern web browser can be used to access and run the web application.
- Internet Connection: Required for installing software packages, accessing development resources, and performing database-related operations when required.

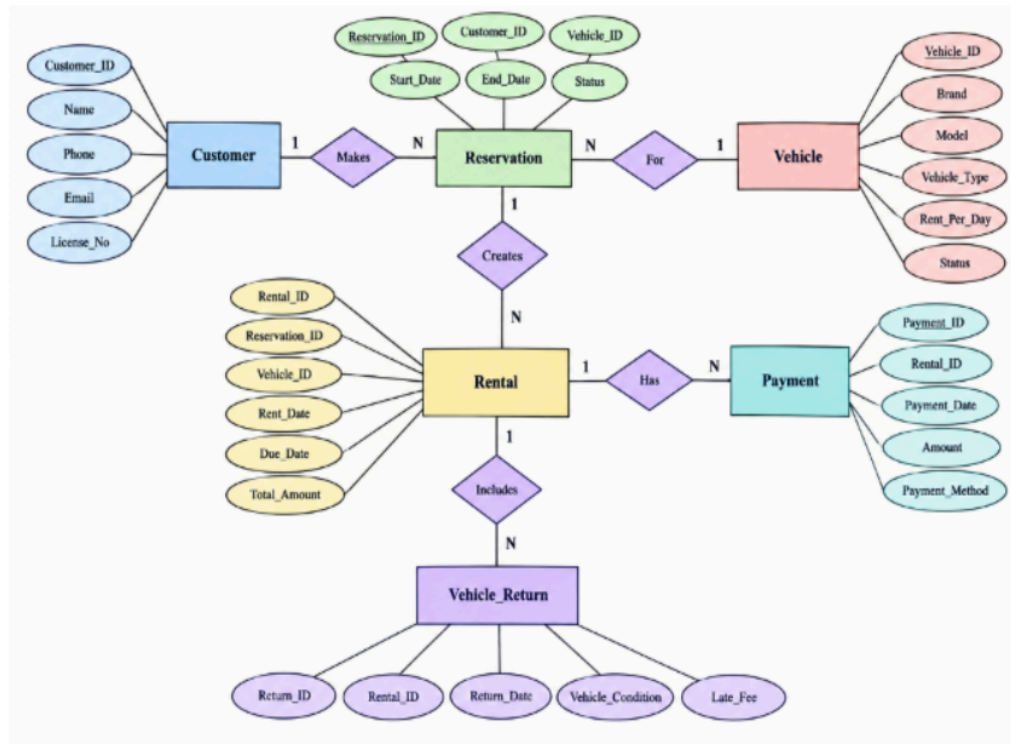
Hardware Requirements

- Processor: Intel Core i3 or equivalent and above

- RAM: Minimum 4 GB, preferably 8 GB for smooth development and execution.
- Storage: At least 10 GB of available storage for the application, database, development tools, and project files.
- Display: Standard laptop or desktop display suitable for developing and accessing the web application.
- Keyboard and Mouse: Required for coding, database management, and system interaction.
- Internet Connectivity: Required for downloading required packages, accessing online resources, and performing necessary development activities.

Overall, the Car Rental Management System can be developed and executed using a standard laptop or desktop computer with a modern web browser, Python and Flask development environment, and MySQL database. The required hardware and software are commonly available, making the system suitable for development and demonstration in a normal academic environment.

ER DIAGRAM



RELATIONAL SCHEMA AND NORMALIZATION

Relational Schema

The relational schema defines the structure of the database by specifying the tables, attributes, primary keys, and foreign keys used in the Car Rental Management System. The database consists of six main relations: CUSTOMER, VEHICLE, RESERVATION, RENTAL, VEHICLE_RETURN, and PAYMENT.

CUSTOMER

The CUSTOMER table stores customer details. Customer_ID uniquely identifies each customer, while License_No is kept unique.

```
CUSTOMER(  
  Customer_ID PK,  
  Name,  
  Phone,  
  Email,  
  License_No UNIQUE  
)
```

VEHICLE

The VEHICLE table stores vehicle details such as brand, model, type, daily rent, and availability status.

```
VEHICLE(  
  Vehicle_ID PK,  
  Brand,  
  Model,  
  Vehicle_Type,  
  Rent_Per_Day,  
  Status  
)
```

RESERVATION

The RESERVATION table records customer reservations and connects customers and vehicles using foreign keys.

```
RESERVATION(  
Reservation_ID PK,  
Customer_ID FK,  
Vehicle_ID FK,  
Start_Date,  
End_Date,  
Status  
)
```

RENTAL

The RENTAL table stores rental transaction details, including the rental date, due date, and total amount.

```
RENTAL(  
Rental_ID PK,  
Reservation_ID FK,  
Vehicle_ID FK,  
Rent_Date,  
Due_Date,  
Total_Amount  
)
```

VEHICLE_RETURN

The VEHICLE_RETURN table stores vehicle return details, including return date, vehicle condition, and late fee.

```
VEHICLE_RETURN(  
Return_ID PK,  
Rental_ID FK,  
Return_Date,  
Vehicle_Condition,  
Late_Fee  
)
```

PAYMENT

The PAYMENT table stores payment details associated with rental transactions.

```
PAYMENT(  
Payment_ID PK,  
Rental_ID FK,  
Payment_Date,  
Amount,  
Payment_Method  
)
```

Normalization

Normalization is the process of organizing data to reduce **data redundancy** and improve **data consistency**. The database in this project is structured using normalized relations.

- First Normal Form — 1NF

A table is in **1NF** when all attributes contain atomic values and there are no repeating groups. In this project, each field stores a single value and every record has a unique primary key. Therefore, the tables satisfy **1NF**.

- Second Normal Form — 2NF

A table is in **2NF** when it is already in 1NF and all non-key attributes depend on the complete primary key. In this project, each table uses a separate primary key such as Customer_ID, Vehicle_ID, Reservation_ID, and Rental_ID. Therefore, the relations satisfy **2NF**.

- Third Normal Form — 3NF

A table is in **3NF** when it is in 2NF and non-key attributes do not depend on other non-key attributes. In this project, customer details are stored only in the CUSTOMER table and vehicle details are stored only in the VEHICLE table. Other tables use foreign keys to refer to them instead of duplicating the information. Therefore, the database satisfies **3NF**.

Normalization helps the project **reduce duplication, improve consistency, simplify updates, and maintain data integrity** between the related tables.

DATA DICTIONARY

The data dictionary describes the structure of the database tables used in the Car Rental Management System. It includes the field names, data types, and constraints such as primary keys, foreign keys, and unique values

Table	Field	Data Type	Constraint
CUSTOMER	Customer_ID	INT	Primary Key, Auto Increment
	Name	VARCHAR(100)	NOT NULL
	Phone	VARCHAR(15)	NOT NULL
	Email	VARCHAR(100)	NOT NULL
	License_No	VARCHAR(30)	UNIQUE

RENTAL	Rental_ID	INT	Primary Key, Auto Increment
	Reservation_ID	INT	Foreign Key
	Vehicle_ID	INT	Foreign Key
	Rent_Date	DATE	NOT NULL
	Due_Date	DATE	NOT NULL
	Total_Amount	DECIMAL(10,2)	NOT NULL

VEHICLE_RETURN	Return_ID	INT	Primary Key, Auto Increment
	Rental_ID	INT	Foreign Key
	Return_Date	DATE	NOT NULL
	Vehicle_Condition	VARCHAR(100)	Optional
	Late_Fee	DECIMAL(10,2)	Default 0.00

PAYMENT	Payment_ID	INT	Primary Key, Auto Increment
	Rental_ID	INT	Foreign Key
	Payment_Date	DATE	NOT NULL
	Amount	DECIMAL(10,2)	NOT NULL
	Payment_Method	VARCHAR(30)	NOT NULL

VEHICLE	Vehicle_ID	INT	Primary Key, Auto Increment
	Brand	VARCHAR(50)	NOT NULL
	Model	VARCHAR(50)	NOT NULL
	Vehicle_Type	VARCHAR(30)	NOT NULL
	Rent_Per_Day	DECIMAL(10,2)	NOT NULL
	Status	VARCHAR(20)	NOT NULL



RESERVATION	Reservation_ID	INT	Primary Key, Auto Increment
	Customer_ID	INT	Foreign Key
	Vehicle_ID	INT	Foreign Key
	Start_Date	DATE	NOT NULL
	End_Date	DATE	NOT NULL
	Status	VARCHAR(20)	NOT NULL

SOL COMMANDS

DDL COMMAND

DDL (Data Definition Language) commands are used to create and define the structure of the database and its tables. In this project, DDL is used for creating the Car Rental database and the tables required to store customer, vehicle, reservation, rental, return, and payment information.

CODE:

```
mysql> CREATE TABLE Customer (  
Customer_ID INT PRIMARY KEY,  
Name VARCHAR(50), Phone VARCHAR(15),  
Email VARCHAR(50),  
License_No VARCHAR(20) UNIQUE );
```

OUTPUT:

Field	Type	Null	Key	Default	Extra
Customer_ID	int	NO	PRI	NULL	
Name	varchar(50)	YES		NULL	
Phone	varchar(15)	YES		NULL	
Email	varchar(50)	YES		NULL	
License_No	varchar(20)	YES	UNI	NULL	

5 rows in set (0.04 sec)

DQL COMMAND

DML (Data Manipulation Language) commands are used to manage the records stored in these tables. In this project, DML is used to add and modify customer and rental-related information.

CODE:

```
mysql> UPDATE Vehicle SET Status = 'Rented' WHERE Vehicle_ID = 1;
```

OUTPUT:

```
mysql> SELECT * FROM Vehicle;
+-----+-----+-----+-----+-----+-----+
| Vehicle_ID | Brand  | Model  | Vehicle_Type | Rent_Per_Day | Status  |
+-----+-----+-----+-----+-----+-----+
|          1 | Toyota | Innova | SUV          |      2500.00 | Rented |
|          2 | Honda  | City   | Sedan        |      2000.00 | Available |
|          3 | Hyundai | Creta  | SUV          |      2200.00 | Available |
|          4 | Tata   | Nexon  | SUV          |      1800.00 | Rented |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

QUERIES WITH OUTPUTS

Query 1: SELECT – Display all customers

```
mysql> SELECT * FROM Customer;
+-----+-----+-----+-----+-----+
| Customer_ID | Name   | Phone   | Email                | License_No |
+-----+-----+-----+-----+-----+
|          1 | Sneha  | 9876543210 | sneha@gmail.com      | DL12345    |
|          2 | Rahul  | 9876501234 | rahul@gmail.com      | DL12346    |
|          3 | Ananya | 9988776655 | ananya@gmail.com     | DL12347    |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Query 2: WHERE – Display available vehicles

```
mysql> SELECT * FROM Vehicle
-> WHERE Status = 'Available';
```

Vehicle_ID	Brand	Model	Vehicle_Type	Rent_Per_Day	Status
1	Toyota	Innova	SUV	2500.00	Available
3	Hyundai	Creta	SUV	2200.00	Available

```
2 rows in set (0.00 sec)
```

Query 3: JOIN – Display customer reservation details

```
mysql> SELECT c.Name, r.Reservation_ID, r.Start_Date, r.End_Date
-> FROM Customer c
-> JOIN Reservation r
-> ON c.Customer_ID = r.Customer_ID;
```

Name	Reservation_ID	Start_Date	End_Date
Sneha	1	2026-10-10	2026-10-12
Rahul	2	2026-10-15	2026-10-17
Ananya	3	2026-10-20	2026-10-22

```
3 rows in set (0.00 sec)
```

Query 4: GROUP BY / Aggregate – Count vehicles by type

```
mysql> SELECT Vehicle_Type, COUNT(*) AS Total_Vehicles
-> FROM Vehicle
-> GROUP BY Vehicle_Type;
```

Vehicle_Type	Total_Vehicles
SUV	3
Sedan	2
Hatchback	1

```
3 rows in set (0.00 sec)
```

Query given during presentation-II

QUESTION:

Retrieve the details of the customer named Sneha who rented an Innova car and made the payment using cash.

CODE:

```
SELECT c.Name, v.Brand, v.Model, p.Amount, p.Payment_Method FROM  
Customer c
```

```
JOIN Rental r ON c.Customer_ID = r.Reservation_ID
```

```
JOIN Vehicle v ON r.Vehicle_ID = v.Vehicle_ID
```

```
JOIN Payment p ON r.Rental_ID = p.Rental_ID
```

```
WHERE c.Name = 'Sneha'
```

```
AND v.Model = 'Innova'
```

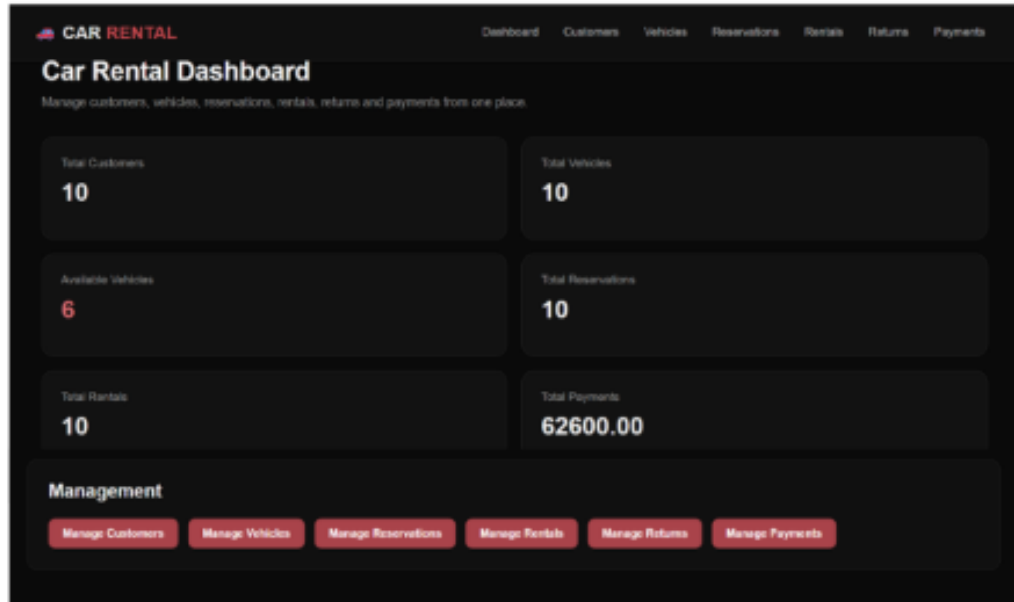
```
AND p.Payment_Method = 'Cash';
```

OUTPUT:

```
mysql> SELECT c.Name, v.Brand, v.Model, p.Amount, p.Payment_Method  
-> FROM Customer c  
-> JOIN Rental r ON c.Customer_ID = r.Reservation_ID  
-> JOIN Vehicle v ON r.Vehicle_ID = v.Vehicle_ID  
-> JOIN Payment p ON r.Rental_ID = p.Rental_ID  
-> WHERE c.Name = 'Sneha'  
-> AND v.Model = 'Innova'  
-> AND p.Payment_Method = 'Cash';  
  
+-----+-----+-----+-----+-----+  
| Name | Brand | Model | Amount | Payment_Method |  
+-----+-----+-----+-----+-----+  
| Sneha | Toyota | Innova | 2500.00 | Cash           |  
+-----+-----+-----+-----+-----+  
1 row in set (0.00 sec)
```

UI DESIGN

Dashboard



Customer records

The 'CUSTOMERS' section has a subtitle 'Manage customer information and driving license details.' and an 'ADD CUSTOMER' link. Below is a form with five input fields: Customer ID, Name, Phone, Email, and License No. A '+ ADD CUSTOMER' button is positioned below the form.

The 'CUSTOMER RECORDS' section displays a table with the following data:

Customer ID	Name	Phone	Email	License No	Actions
101	Rahul	9876543210	rahul@gmail.com	DL1001	<button>EDIT</button> <button>DELETE</button>
102	Priya	9876543211	priya@gmail.com	DL1002	<button>EDIT</button> <button>DELETE</button>
103	Arjun	9876543212	arjun@gmail.com	DL1003	<button>EDIT</button> <button>DELETE</button>

01. VIEW OPERATION

CAR RENTAL						Dashboard	Customers	Vehicles	Reservations	Rentals	Returns	Payments
Customer ID	Name	Phone	Email	License No	Actions							
101	Rahul	9876543210	rahul@gmail.com	DL1001	EDIT	DELETE						
102	Priya	9876543211	priya@gmail.com	DL1002	EDIT	DELETE						
103	Ajay	9876543212	ajay@gmail.com	DL1003	EDIT	DELETE						
104	Sneha	9876543213	sneha@gmail.com	DL1004	EDIT	DELETE						
105	Kiran	9876543214	kiran@gmail.com	DL1005	EDIT	DELETE						
106	Ananya	9876543215	ananya@gmail.com	DL1006	EDIT	DELETE						
107	Rishi	9876543216	rishi@gmail.com	DL1007	EDIT	DELETE						

02.INSERTION OPERATION

CAR RENTAL

DashboardCustomersVehiclesReservationsRentalsReturnsPayments

VEHICLE RETURNS
Manage returned vehicles, conditions and late fees.
ADD VEHICLE RETURN

Return ID

Rental ID

Return Date

Vehicle Condition

Late Fee

101

401

2024-09-15

Good

0

ADD RETURN

RETURN RECORDS

Return ID	Rental ID	Return Date	Vehicle Condition	Late Fee	Actions
101	401	2024-09-15	Good	0	EDITDELETE
102	402	2024-09-20	Good	100.00	EDITDELETE
103	403	2024-09-25	Good	0	EDITDELETE

CAR RENTAL

DashboardCustomersVehiclesReservationsRentalsReturnsPayments

Vehicle return recorded successfully

VEHICLE RETURNS
Manage returned vehicles, conditions and late fees.
ADD VEHICLE RETURN

Return ID

Rental ID

Return Date

Vehicle Condition

Late Fee

2024-09-20

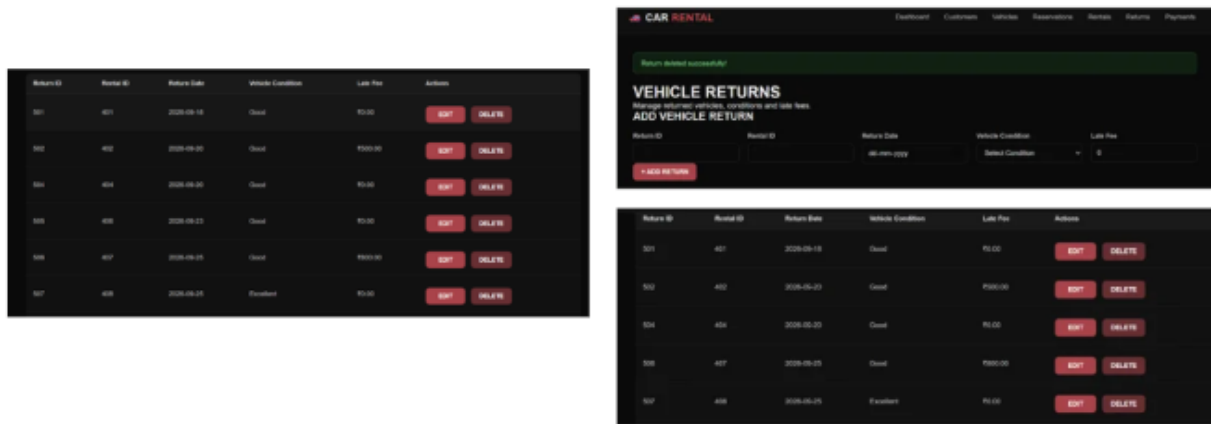
Select Condition

0

ADD RETURN

104	404	2024-09-25	Good	0	EDITDELETE
105	405	2024-09-28	Minor Damage	1500.00	EDITDELETE
106	406	2024-10-02	Damaged	1500.00	EDITDELETE
107	407	2024-07-09	Good	0	EDITDELETE

03. DELETION OPERATION



IMPLEMENTATION DETAILS

Technology Stack

The Car Rental Management System is implemented as a web-based database application using a combination of frontend, backend, and database technologies. HTML5 is used to create the structure of the web pages, while CSS3 is used for styling and designing the user interface. JavaScript is used wherever required to provide basic interaction and improve the user experience.

The backend of the system is developed using Python with the Flask framework. Flask handles the application requests and connects the web interface with the database. MySQL is used as the relational database management system for storing customer, vehicle, reservation, rental, vehicle return, and payment records. MySQL Connector/Python is used to establish communication between the Python application and MySQL. Visual Studio Code is used as the development environment.

Technology	Purpose
HTML5	Designing the structure of web pages
CSS3	Styling and formatting the interface
JavaScript	Adding basic client-side interaction
Python	Backend application development
Flask	Connecting the web application with the backend
MySQL	Storing and managing project data
MySQL Connector/Python	Connecting Python with MySQL
Visual Studio Code	Development environment

Database Connectivity

Database connectivity is an important part of the system because the web application needs to communicate with the MySQL database. The Flask application uses MySQL Connector/Python to establish a connection with the database. Once the connection is established, SQL queries can be executed through Python to insert, retrieve, update, and delete records.

A basic database connection is shown below:

```
import mysql.connector
```

```
db = mysql.connector.connect(  
    host="localhost",  
    user="root",  
    password="your_password",  
    database="CarRentalDB"  
)
```

```
cursor = db.cursor()
```

The connection object is used to communicate with the MySQL database, while the cursor is used to execute SQL commands. After operations such as insertion or updating, `commit()` is used to save the changes permanently in the database.

Key Code Snippets

One of the important operations in the project is adding a new customer to the database. The following Python code executes an INSERT query using values received from the web application.

```
query = """  
INSERT INTO Customer  
(Name, Phone, Email, License_No)  
VALUES (%s, %s, %s, %s)  
"""
```

```
values = (name, phone, email, license_no)
```

```
cursor.execute(query, values)  
db.commit()
```

The use of parameterized values helps in passing user information safely to the SQL query. Similar database operations are implemented for vehicles, reservations,

rentals, returns, and payments. The application also uses SELECT, UPDATE, and DELETE queries to retrieve and manage existing records.

TESTING RESULT

Testing was performed to verify that the Car Rental Management System functions correctly and that the database operations produce the expected results. Each major feature was tested by providing suitable input and comparing the actual result with the expected result.

The testing mainly covers customer management, vehicle management, reservations, rentals, vehicle returns, payments, and CRUD operations. The purpose of testing is to identify errors and ensure that the application and database work together correctly.

Test Case	Test Description	Expected Result	Actual Result	Status
TC01	Add a new customer	Customer details should be stored in the database	Customer record stored successfully	Pass
TC02	View customer details	Existing customer records should be displayed	Customer details displayed correctly	Pass
TC03	Add a new vehicle	Vehicle details should be added to the database	Vehicle record added successfully	Pass
TC04	Update vehicle status	Vehicle status should change according to the operation	Vehicle status updated successfully	Pass

TC05	Delete a customer record	Selected customer record should be removed	Customer record deleted successfully	Pass
------	--------------------------	--	--------------------------------------	------

CONCLUSION

The **Car Rental Management System** was developed to provide a simple and organized solution for managing the major activities of a car rental service. The system maintains important information related to customers, vehicles, reservations, rentals, vehicle returns, and payments in a centralized MySQL database.

The project demonstrates the practical implementation of important **DBMS concepts** such as relational tables, primary keys, foreign keys, constraints, normalization, SQL queries, CRUD operations, and database connectivity. The Flask-based web interface provides an easier way to interact with the database instead of entering every operation manually through the MySQL command line.

The system helps reduce duplicate data, improves data organization, and makes it easier to retrieve and update rental information. Overall, the project provides practical experience in connecting a web application with a relational database and applying DBMS concepts to a real-world problem.

FUTURE ENHANCEMENTS

Although the current system provides the main functions required for managing a car rental service, several features can be added in the future. **Online payment integration** can be implemented to allow customers to make payments directly through the system. A **customer login and authentication system** can also be added to provide secure access to personal reservations and rental information.

The system can further be enhanced with **advanced vehicle search and filtering**, allowing customers to search vehicles based on type, availability, model, and rental price. Automatic calculation of **rental charges and late fees** can also reduce manual calculations. Email or SMS notifications can be introduced for reservation confirmations, due dates, and vehicle return reminders.

In the future, the system can also include an **admin dashboard with reports and graphical summaries** of rentals, payments, available vehicles, and customer activity.

REFERENCES

1. **MySQL Documentation** – MySQL Reference Manual for SQL commands, database creation, table management, constraints, and queries.
2. **Python Documentation** – Python Software Foundation documentation for Python programming concepts and database connectivity.
3. **Flask Documentation** – Official Flask documentation for understanding web application development, routing, and backend implementation.
4. **W3Schools** – SQL, HTML, CSS, JavaScript, and Python tutorials used for understanding basic programming and database concepts.
5. **GeeksforGeeks** – Reference material for DBMS concepts, SQL queries, normalization, joins, and database operations.
6. **MySQL Connector/Python Documentation** – Reference for establishing and managing connections between Python applications and MySQL databases.

GitHub Repository Link

<https://github.com/Sansita00/DBMS-Course-Project>